

Seminar Report

Team Members: Group 32

Abdur Rehman Khalid

Arnab Kumar Ghosh

Dip Chowdhury

Muhammad Umair

Pradip Kumar Das

Zihao Yang

March 2, 2026

Contents

1	Question 1: Batch vs. Streaming Processing	2
2	Question 2: Data Formats and Storage	2
3	Question 3: The Big Data Ecosystem – Is There a Logic?	2
3.1	The Volume Problem: Disk-Bound Batch Processing	3
3.2	The Random Access Problem: Wide-Column Stores	3
3.3	The Velocity Problem: The Streaming Paradigm	3
3.4	The Iteration Problem: In-Memory Unification	3
3.5	Modern Convergence: Cloud-Native and Lakehouse Architectures	3
4	Question 4: The Rise of NoSQL	3
5	Question 5: Scalability – Theory and Practice	3

1 Question 1: Batch vs. Streaming Processing

From a problem perspective, the choice between batch and stream processing is dictated by the nature of the data and latency requirements of the application. Batch processing assumes the existence of static, bounded datasets where all input data are available before the computation begins [1]. This is necessary for scenarios where system throughput and computation is preferred over system latency, such as training language models, analyzing large scale documents where maximizing overall system output and robust fault tolerance are more critical than response time. On the other hand, streaming processing is required for continuous and unbounded data such as server log monitoring, internet-of-things (IoT) sensor outputs or social media where analytics need to be performed continuously in real-time. For example, Twitter uses real-time streaming to measure real-time active user count (RTAC) indicator to find user engagement events [2] where low latency is a strict necessity.

Early batch frameworks like Google’s MapReduce [3] achieved massive scalability and fault-tolerance by using high-throughput disk-based storage. While this ensures reliability against machine failures, it incurs significant disk I/O latency overhead, making the framework unsuitable for real-time queries. To tackle the latency issue, Apache Storm was introduced, which process data one at a time using a directed graph [4]. To address the limitations of Storm, Twitter developed Heron which adopted a one task per JVM model to achieve process isolation and a backpressure mechanism. However, early streaming systems often traded off the consistency and high throughput characteristic of batch processing systems making the latter still relevant.

Some recent frameworks have been developed to connect the gap between batch and streaming processing by unifying the architectural division. Apache Spark approaches this unification through ”micro-batching”, dividing the continuous streams into small batches to leverage fault-tolerance and in-memory batch execution [1]. While Apache Flink took the inverse approach by treating batch processing as streaming processing where data stream is considered finite [5]. Further blurring the lines between the two paradigms, abstraction model like Apache Beam [6] allows programmer to write unified and portable data processing pipelines that can be executed on either batch or streaming processing engines.

Finally, the Dataflow Model [7] serves as the definitive framework for unifying batch and streaming processing to achieve batch processing correctness within a streaming processing environment. Ultimately, the model suggests a trade-off between correctness, latency, and cost, allowing a single pipeline to produce both low-latency high-accuracy results, effectively treating batch processing as a specific instance of streaming where the data source happens to be finite.

2 Question 2: Data Formats and Storage

3 Question 3: The Big Data Ecosystem – Is There a Logic?

The big data ecosystem is not fragmented. It is a coherent, reactive progression. Each generation of technologies emerged to solve the critical bottlenecks—volume, latency, or system coupling—of its predecessors.

3.1 The Volume Problem: Disk-Bound Batch Processing

Traditional relational databases could not scale horizontally. Google introduced GFS and MapReduce [3], later open-sourced as Apache Hadoop and HDFS, to handle massive data volumes on commodity hardware. But Hadoop had a fundamental flaw: aggressive disk I/O. MapReduce wrote intermediate results to disk after every computation phase. This made it highly fault-tolerant but terrible for low-latency queries and iterative machine learning [8].

3.2 The Random Access Problem: Wide-Column Stores

HDFS was optimized for sequential scans of large files. It could not serve real-time point queries. Google BigTable [9] solved this as a distributed NoSQL layer over GFS. This inspired Apache HBase for Hadoop. Facebook then built Apache Cassandra to eliminate HBase's centralized master node. Cassandra combined BigTable's column-family model with Dynamo's masterless architecture. It traded strong consistency for extreme availability and zero single points of failure.

3.3 The Velocity Problem: The Streaming Paradigm

As data velocity increased, batch processing became too slow. Twitter Storm pioneered distributed streaming but had severe centralized bottlenecks. Twitter Heron redesigned Storm to fix this by decoupling the scheduling framework. Meanwhile, LinkedIn introduced Apache Kafka [10] to prevent high-velocity data from overwhelming processing engines. Kafka acted as a distributed log broker, decoupling data ingestion from computation.

3.4 The Iteration Problem: In-Memory Unification

Apache Spark [8] directly addressed Hadoop's disk bottleneck with Resilient Distributed Datasets (RDDs). This enabled fault-tolerant, in-memory processing that accelerated iterative algorithms by orders of magnitude. Spark tried to unify batch and streaming through micro-batching, but this still added latency. Apache Flink countered this by treating batch as bounded streams, achieving true low-latency processing.

3.5 Modern Convergence: Cloud-Native and Lakehouse Architectures

Today's architectures reject the tight coupling of storage and compute that defined Hadoop. Modern systems use cheap, scalable cloud object storage with elastic compute engines. This evolution led to the Data Lakehouse [11], powered by formats like Apache Iceberg. The Lakehouse merges data lakes and warehouses, bringing ACID transactions and schema enforcement to decoupled cloud storage.

4 Question 4: The Rise of NoSQL

5 Question 5: Scalability – Theory and Practice

References

- [1] S. Shahrivari, “Beyond batch processing: Towards real-time and streaming big data,” *Computers*, vol. 3, no. 4, pp. 117–129, 2014. [Online]. Available: <https://www.mdpi.com/2073-431X/3/4/117>
- [2] S. Kulkarni, N. Bhagat, M. Fu, V. Kedigehalli, C. Kellogg, S. Mittal, J. M. Patel, K. Ramasamy, and S. Taneja, “Twitter heron: Stream processing at scale,” in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 239–250. [Online]. Available: <https://doi.org/10.1145/2723372.2742788>
- [3] J. Dean and S. Ghemawat, “Mapreduce: Simplified data processing on large clusters,” in *Proceedings of the 6th Conference on Symposium on Operating Systems Design & Implementation*. USENIX Association, 2004, pp. 10–10.
- [4] A. Toshniwal, S. Taneja, A. Shukla, K. Ramasamy, J. M. Patel, S. Kulkarni, J. Jackson, K. Gade, M. Fu, J. Donham, N. Bhagat, S. Mittal, and D. Ryaboy, “Storm@twitter,” in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 147–156. [Online]. Available: <https://doi.org/10.1145/2588555.2595641>
- [5] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache flink™: Stream and batch processing in a single engine,” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 36, no. 4, pp. 28–38, 2015.
- [6] S. Li, P. Gerver, J. MacMillan, D. Debrunner, W. Marshall, and K.-L. Wu, “Challenges and experiences in building an efficient apache beam runner for ibm streams,” *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1742–1754, 2018.
- [7] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, “The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proc. VLDB Endow.*, vol. 8, no. 12, p. 1792–1803, Aug. 2015. [Online]. Available: <https://doi.org/10.14778/2824032.2824076>
- [8] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*. USENIX Association, 2012, pp. 2–2.
- [9] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, “Bigtable: A distributed storage system for structured data,” in *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*. USENIX Association, 2006, pp. 205–218.

- [10] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, vol. 11, 2011, pp. 1–7.
- [11] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, “Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics,” in *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)*, 2021.